TimeAudit 快速部署 & 容灾手册

这份文档教你把 TimeAudit **从零装起来、搬到新电脑**、或者**系统崩了之后恢复。** 每一步都给"**先复制这条命令**，再用一句话解释它干了啥"，照着抄就行。看不懂术语没关系，按顺序执行即可。

> 想先搞懂这个项目是什么，看 **README.md**；装好后想看懂 6 张仪表盘怎么用、怎么靠数据判断电脑问题，看 **使用手册.md**（78 个面板逐个精讲）。

你属于哪种情况？直接跳到对应章节

你的情况	看哪一章
🆕 全新电脑，第一次装，还没有任何历史数据	**场景 A：全新安装**
🔁 换了台新电脑，想把旧电脑的历史数据一起搬过去	**场景 B：换电脑迁移**
💥 系统崩溃/重装/硬盘坏了，想用之前的备份恢复	**场景 C：容灾恢复**
🛠️ 已经装好了，只想做日常维护（看日志/重启/备份）	**第 6 章：日常运维**

---

0. 开始之前：你需要知道的几个固定值

整个系统的"户口本"，全程别改：

项目	值
项目根目录	`E:\Projects\Tools\TimeAudit`（强烈建议就放这，脚本里很多写死的绝对路径）
数据库（PostgreSQL）	地址 `localhost:45432`，库名 `time_audit`；口令由当前用户环境变量 `TIMEAUDIT_DB_PASSWORD` 注入，不进入 Git
大盘（Grafana）	浏览器开 `http://localhost:43000`
开机自启计划任务名	`TimeAudit_AutoStart`
硬件参考	Windows 11 + AMD 9950X3D + NVIDIA GeForce RTX 5090 D（其它配置也能跑，硬件细节自动适配）

两个外部 exe 已经随项目附带，不用单独下载：`LibreHardwareMonitor.exe`（读 CPU/GPU 真实电压温度）、`PresentMonConsole.exe`（RTSS 共享内存不可用时的 fallback）。常用游戏或唯一桌面动画帧率优先来自已运行 RTSS 的官方共享内存。

场景 A：全新安装

一共 6 步，从装环境到开机自启。请全程使用**管理员权限的 PowerShell**（开始菜单搜 PowerShell → 右键"以管理员身份运行"）。

A1. 装 WSL2（Docker 的底座）

Docker Desktop 在 Windows 上要靠 WSL2 跑 Linux 容器。

```
# 开启"Linux 子系统"和"虚拟机平台"两个 Windows 组件
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux -NoRestart
Enable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform -NoRestart
# 升级 WSL 内核到最新
wsl --update
```

> ⚠️ **做完这步必须重启一次电脑**，虚拟机底座才会真正生效。重启后继续 A2。

A2. 装 Docker Desktop 和 Python（用 winget 静默装）

```
# 装 Docker Desktop
winget install Docker.DockerDesktop --silent --accept-package-agreements --accept-source-agreements
# 装 Python 3.11（和容器内环境对齐；3.10~3.12 都行）
winget install Python.Python.3.11 --silent --accept-package-agreements

# 让新装的命令在当前窗口立刻可用（免得重开终端）
$env:Path = [System.Environment]::GetEnvironmentVariable("Path","Machine") + ";" +
[System.Environment]::GetEnvironmentVariable("Path","User")

python --version   # 验证
docker --version   # 验证
```

> 装完 Docker Desktop 后**打开它一次**，等右下角小鲸鱼图标变绿（引擎就绪）。 CN 如果在国内拉镜像总超时：打开 Docker Desktop → Settings → Resources → Proxies， 手动填上你的本地代理（如 `http://127.0.0.1:7892`），Apply & restart。

A3. 装宿主机 Python 依赖

把项目放到 `E:\Projects\Tools\TimeAudit`，然后：

```
cd E:\Projects\Tools\TimeAudit
python -m pip install --upgrade pip
pip install -r requirements.txt
```

这会装 3 个库：`psutil`（系统采集）、`nvidia-ml-py`（读显卡，import 名是 pynvml）、`asyncpg`（异步写数据库）。**不需要** pywin32——引擎底层调用全走 Python 自带的 ctypes。

A4. 起数据库 + 大盘（Docker 三容器）

```
cd E:\Projects\Tools\TimeAudit
docker compose up -d --build

# 等几秒，确认三个容器都 Up
Start-Sleep -Seconds 5
docker compose ps
```

你应该看到 `audit-postgres`、`audit-ingester`、`audit-grafana` 三个都在 `Up`。

> 💡 `docker-compose.yml` 已内置 PostgreSQL 性能调优（`shared_buffers=2GB` / `work_mem=16MB` / `effective_cache_size=8GB` + NVMe 友好的 planner 参数）和 `shm_size: '512mb'`（并行查询大分区时防 `/dev/shm` 不足报错），**起容器时自动生效，无需手动配 postgresql.conf**。日后想调这些值，编辑 compose 后 `docker compose up -d audit-db` 重建容器即可。

A5. 建表（全新库的关键一步，别漏！）

全新的数据库是**空的**，引擎只会自动建"分区子表"，但父表得先手动建好。跑一次 `schema.sql`：

```
cmd /c "docker exec -i audit-postgres psql -U leyang -d time_audit <
E:\Projects\Tools\TimeAudit\schema.sql"
```

> 用 `cmd /c "... < 文件"` 是因为 PowerShell 不支持 `<` 这个输入重定向符号。 验证建好了（应列出 6 张表）：
> `powershell docker exec -i audit-postgres psql -U leyang -d time_audit -c "\dt public.*"`

A6. 配开机自启（提权 + 隐身，闭环最关键的一步）

引擎必须以**管理员权限**运行（RTSS 首选读取本身只读，但 PresentMon fallback 与部分 LHM/NVML 路径仍需要提权）。 下面用 Windows 计划任务实现"开机自动、隐身、提权"，并且**绕开两个常见坑**：

- 不要用 `SYSTEM` 账户跑——那样会落到 Session 0，`GetForegroundWindow()` 永远返回空，前台窗口采集直接失明。
- 用 `Interactive`（当前登录用户）+ `Highest`（最高权限），既能抓前台窗口又自带提权，还不会弹 UAC。

```
$Action    = New-ScheduledTaskAction -Execute "E:\Projects\Tools\TimeAudit\start_all.bat" -
WorkingDirectory "E:\Projects\Tools\TimeAudit"
$Trigger   = New-ScheduledTaskTrigger -AtLogOn
$Principal = New-ScheduledTaskPrincipal -LogonType Interactive -RunLevel Highest
# 笔记本用电池也跑、不因断电停、关掉默认 3 天强杀
$Settings  = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries -
ExecutionTimeLimit 0
Register-ScheduledTask -TaskName "TimeAudit_AutoStart" -Action $Action -Trigger $Trigger -Principal
$Principal -Settings $Settings -Force
```

`start_all.bat` 做的事：拉起 AHK 脚本 → `docker compose up -d` → 等数据库端口就绪 → 使用绝对路径的 `pythonw.exe` 配合 `-WorkingDirectory` 隐形拉起 `main.py`（**这里在脚本中已通过绝对物理路径与指定工作目录防范了自启闪退的硬伤**，因为计划任务在 Highest 权限运行时并不包含用户级 PATH 环境变量，若不用绝对路径和显式工作目录，会导致 Python 进程加载依赖失败直接静默闪退）。

立即手动跑一次（不用等下次开机）：

```
schtasks /run /tn TimeAudit_AutoStart
```

A7. 验证一切正常

```
# 等 15 秒让引擎采几拍，然后体检
Start-Sleep -Seconds 15
python E:\Projects\Tools\TimeAudit\test_telemetry_health.py
```

大部分项打勾就成了。然后浏览器开 `http://localhost:43000` 看大盘。

> 💡 **关于 LibreHardwareMonitor**：它由 Windows 计划任务 `LibreHardwareMonitor` 作为唯一运行时 owner 启动，外部 `telemetry_watchdog.ps1` 负责端点恢复；TimeAudit 引擎只读取它，**不要**手动再开第二个实例。 它读数走网页服务 `http://127.0.0.1:18085/data.json`（端口写在 `LibreHardwareMonitor.config` 的 `listenerPort`）。 如果体检报告说"LHM 电压没入库"，先确认该计划任务只有一个实例、web server 已开启且端口是 18085；服务不可达时等待外部 watchdog 恢复，不要用手动启动叠加实例。

场景 B：换电脑迁移（带历史数据）

目标：把旧电脑的 TimeAudit **连同历史数据**搬到新电脑。

B1. 旧电脑：备份数据库 + Grafana 仪表盘

```
cd E:\Projects\Tools\TimeAudit
powershell -ExecutionPolicy Bypass -File .\backup_all.ps1
```

这一条会同时备份**数据库**（→ `G:\80_Backup\TimeAudit\postgresql\time_audit_*.dump`）和 **Grafana 仪表盘**（仪表盘 JSON → `grafana_dashboards\`，是进 git 的"前端代码"；`grafana.db` 二进制 → `G:\80_Backup\TimeAudit\grafana_db`）。

B2. 把整个项目拷到新电脑

把 **整个** `E:\Projects\Tools\TimeAudit` **文件夹**拷到新电脑的 `E:\Projects\Tools\TimeAudit`，并把 `G:\80_Backup\TimeAudit` 一并复制到新电脑的 G 盘（U 盘/网盘/局域网都行）。 **至少要包含**：所有 `.py`、`.ps1`、`docker-compose.yml`、`Dockerfile`、`schema.sql`、 两个 `.exe`、`grafana_provisioning/`、`grafana_dashboards/`（仪表盘代码），以及 G 盘刚生成的两类备份。

> `postgres_data/` 这个文件夹**可以不拷**（数据靠 B1 的 .dump 恢复更可靠）。 直接拷 `postgres_data/` 也行，但容器没干净关闭时它可能损坏，所以推荐用 .dump。

B3. 新电脑：装好环境 + 起空库

在新电脑上把**场景 A 的 A1 ~ A4 做一遍**（装 WSL2 / Docker / Python / 依赖，然后 `docker compose up -d --build` 起容器）。

> 这一步起来的是个**空数据库**。下一步用备份把数据灌进去——所以 **A5 建表可以跳过**（恢复 .dump 时会自动把表结构一起带进来）。

B4. 新电脑：恢复数据

```
# 等数据库就绪
Start-Sleep -Seconds 5
# 把备份导入（--clean --if-exists 会先清掉同名对象再建，保证干净）
$dump = (Get-ChildItem G:\80_Backup\TimeAudit\postgresql\time_audit_*.dump | Sort-Object LastWriteTime -
Desc | Select-Object -First 1).FullName
cmd /c "docker exec -i audit-postgres pg_restore -U leyang -d time_audit --clean --if-exists < `"$dump`""
```

验证数据回来了：

```
docker exec -i audit-postgres psql -U leyang -d time_audit -c "SELECT count(*) FROM fact_system_hardware;"
```

B5. 新电脑：恢复 Grafana + 配开机自启

1. **恢复 Grafana 仪表盘**：用**场景 C2 第 3 步**的方法（覆盖 `grafana.db` 最省事，或 `python restore_grafana.py`）。
2. 照**场景 A 的 A6** 配好 `TimeAudit_AutoStart` 开机自启，并照 **C1** 重新注册每日备份任务 `TimeAudit_DailyBackup`。

手动各跑一次确认正常，迁移完成。

场景 C：容灾恢复（系统崩了 / 重装 / 硬盘换新）

容灾的前提是**你平时有备份**。所以分两部分：平时怎么自动备份、出事后怎么恢复。

C1. 平时：每天自动备份（已帮你配好）

本项目已经注册好一个计划任务 **`TimeAudit_DailyBackup`**，每天 20:40 自动跑 `backup_all.ps1`， **一次把两样东西都备份了**：

- **PostgreSQL 数据库** → `G:\80_Backup\TimeAudit\postgresql\time_audit_*.dump`（保留最近 14 天）。
- **Grafana 仪表盘** → 导出成 JSON 存 `grafana_dashboards\`（前端"代码"，进 git）并**自动 commit + push 到 GitHub**； 同时复制一份 `grafana.db` 到 `G:\80_Backup\TimeAudit\grafana_db`（完整二进制"数据"兜底，保留 14 份，不进 git）。

想现在立刻手动跑一次全量备份：

```
Start-ScheduledTask -TaskName "TimeAudit_DailyBackup"
# 或者直接： powershell -ExecutionPolicy Bypass -File E:\Projects\Tools\TimeAudit\backup_all.ps1
```

如果将来要在新机器上**重新注册**这个任务（免管理员即可）：

```
$A = New-ScheduledTaskAction -Execute "wscript.exe" -Argument
'"E:\Projects\Tools\TimeAudit\backup_all_hidden.vbs"' -WorkingDirectory "E:\Projects\Tools\TimeAudit"
$T = New-ScheduledTaskTrigger -Daily -At '20:40'
$P = New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME" -LogonType Interactive -RunLevel
Limited
$S = New-ScheduledTaskSettingsSet -StartWhenAvailable -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries
-ExecutionTimeLimit (New-TimeSpan -Hours 2)
Register-ScheduledTask -TaskName "TimeAudit_DailyBackup" -Action $A -Trigger $T -Principal $P -Settings $S
-Force
```

数据库 `.dump` 和 `grafana.db` 默认放在 G 盘，与 E 盘在线数据分开。若这些历史数据对你只是低价值记录， 无需再上复杂异地备份；G 盘损坏时接受丢失即可。Grafana 仪表盘 JSON 仍会自动 push 到 GitHub。

C2. 出事后：恢复

系统重装好之后：

1. 重新装环境 + 起空库：做**场景 A 的 A1 ~ A4**。

2. **恢复数据库**：从 `G:\80_Backup\TimeAudit\postgresql` 选一份 `.dump`，用 **B4** 的命令导入。
3. **恢复 Grafana 仪表盘**（二选一）： - **最省事**：把一份 `grafana.db` 备份覆盖回去 —— 停 Grafana → 拷贝 → 重启，全部仪表盘/用户/偏好原样回来： `powershell docker compose stop audit-dashboard Copy-Item G:\80_Backup\TimeAudit\grafana_db\grafana_<最新时间戳>.db E:\Projects\Tools\TimeAudit\grafana_data\grafana.db -Force docker compose start audit-dashboard` - **或用 JSON 重新导入**（只恢复仪表盘，适合从 GitHub 拉下来的 JSON）： `powershell python E:\Projects\Tools\TimeAudit\restore_grafana.py`
4. 用 **A6** 配好开机自启。

搞定，历史数据零丢失（最多丢上次备份之后的部分）。

> 如果你连备份都没有，但 `postgres_data/` / `grafana_data/` 文件夹还在：可以直接把它们放回原位 再 `docker compose up -d`，容器会直接挂载。但这种方式在容器异常关闭时有损坏风险，不如 `.dump` / `grafana.db` 拷贝可靠。

---

6. 日常运维速查

想干嘛	命令
看引擎实时日志	`Get-Content -Wait -Tail 20 E:\Projects\Tools\TimeAudit\telemetry.log -Encoding utf8`
一键健康体检	`python E:\Projects\Tools\TimeAudit\test_telemetry_health.py`
手动重启引擎	`schtasks /run /tn TimeAudit_AutoStart`
看崩溃守护状态	`Get-ScheduledTaskInfo TimeAudit_Watchdog`（每5分钟自动重启 native 崩溃的引擎；日志 `telemetry_watchdog.log`）
停掉引擎	任务管理器结束 `python.exe`（跑 main.py 的那个），或 `Stop-ScheduledTask -TaskName TimeAudit_AutoStart`
手动全量备份（库+大盘）	`Start-ScheduledTask -TaskName TimeAudit_DailyBackup` 或 `powershell -File E:\Projects\Tools\TimeAudit\backup_all.ps1`
只备份数据库	`powershell -ExecutionPolicy Bypass -File E:\Projects\Tools\TimeAudit\backup_db.ps1`
只备份 Grafana 仪表盘	`python E:\Projects\Tools\TimeAudit\backup_grafana.py`
恢复 Grafana 仪表盘	`python E:\Projects\Tools\TimeAudit\restore_grafana.py`（或覆盖 grafana.db，见场景 C2）
看上次备份结果	`Get-Content E:\Projects\Tools\TimeAudit\log\backup.log -Encoding utf8`
看最近 30 次备份结果	`Get-Content E:\Projects\Tools\TimeAudit\log\backup-history.jsonl -Encoding utf8`（含起止时间、状态、失败环节、退出码）
看容器状态	`docker compose ps`（在 E:\Projects\Tools\TimeAudit 下）
重启数据库/大盘	`docker compose restart`（在 E:\Projects\Tools\TimeAudit 下）
进数据库手敲 SQL	`docker exec -it audit-postgres psql -U leyang -d time_audit`
跑测试套件	`python E:\Projects\Tools\TimeAudit\telemetry_test_suite.py`（应 6/6 全绿）
优化/修复回归测试	`python E:\Projects\Tools\TimeAudit\test_optimizations.py`（应 14/14 全绿）
一句话看懂仪表盘	浏览器开 `http://localhost:43000`，每个面板讲解见 使用手册.md

---

7. 常见问题（FAQ）

Q: 大盘打不开 / 数据库连不上？ 先看容器在不在：`docker compose ps`。不在就 `docker compose up -d`。 Docker Desktop 没启动的话，先打开它等小鲸鱼变绿。

Q: 体检报告说"python 遥测引擎没在运行"？ `schtasks /run /tn TimeAudit_AutoStart` 手动拉一次，再看 `telemetry.log` 有没有新日志。

Q: 跑 Python 脚本报一堆乱码 / `UnicodeEncodeError`？ Windows 默认 GBK 控制台编不了日志里的 emoji。命令前加 `$env:PYTHONUTF8=1` 再跑。

Q: 体检说 FPS / GPU 电压一直是空？ GPU 电压依赖 LHM/NVML；FPS 优先依赖 RTSS 官方共享内存，RTSS 没有有效前台帧时才回退 PresentMon。确认引擎是**通过计划任务提权启动**的，不是手动双击 python 起的；同时确认 RTSS 在游戏启动前已运行。没有游戏时状态应为 `gated_idle`，不能再只凭新鲜零值判断健康。 体检以“最近 30 分钟是否有 FPS 入库样本”来判断帧率采集能力。

Q: `pg_restore` / `psql` 命令里能不能用 PowerShell 的 `<`？ 不能，PowerShell 不支持 `<`。本手册里凡是带 `<` 的都用 `cmd /c "...< 文件"` 包起来了，照抄即可。

Q: 之前装过别的同名计划任务（`TimeAuditEngine` / `TimeAuditTelemetryEngine_Fixed`）怎么办？ 这两个历史任务已禁用，登记为等本人决定，暂不注销；当前启动入口是 `TimeAudit_AutoStart`。

Q: Grafana 仪表盘是怎么自动备份的？我改了大盘要手动导出吗？ 不用手动导出。`TimeAudit_DailyBackup` 会按既有计划自动把非文件预置的仪表盘导出成 JSON 提交到 GitHub， 并复制一份 `grafana.db`。你只管在网页上自由改大盘，改完它自己会在下次备份时记录下来。 网站电脑状态三张看板只维护 `grafana_provisioning/dashboards/` 中的定义，不重复导出和 API 恢复。 想立刻备份就跑 `Start-ScheduledTask -TaskName TimeAudit_DailyBackup`。 自动任务以只读方式从本机 `grafana.db` 取得一致快照，不需要保存或注入 Grafana 口令； 只有显式使用 `backup_grafana.py --source api` 做远程/手动导出时，才从当前进程的私密 `GRAFANA_USER` / `GRAFANA_PASSWORD` 环境变量认证，不写入命令行、文档或 Git。

Q: 数据库密码能改吗？ 能，但要三处一起改：`docker-compose.yml`、`ingest.py`、以及各 `*.py` 里的 `DB_DSN`。改完重建容器。 个人自用、且只监听本机，保持默认也没大问题。

Q: 为什么通过计划任务开机自启提权启动时，主程序没有任何反应或直接闪退？ **A**: 极大可能是因为启动命令没有使用**绝对路径**或者没有指定 **工作目录(WorkingDirectory)**。计划任务在 Highest 权限下运行时，其系统 PATH 环境和普通用户控制台完全不同，它可能找不到用户的 `python` 解释器或 Python 包目录。请检查：1. 动作执行必须使用绝对物理路径（例如 `C:\Users\10979\AppData\Local\Programs\Python\Python311\pythonw.exe E:\Projects\Tools\TimeAudit\main.py`）；2. “起始于”（Start in / Working Directory）选项必须配成绝对路径 `E:\Projects\Tools\TimeAudit`。项目中 `start_all.bat` 已做自愈优化，请确保不要改动其路径格式。

Q: 日志里偶尔出现 PermissionError 拒绝访问 telemetry.log 是什么原因？ **A**: 这是因为有多个实例或后台进程（例如你在控制台手动测试或调试的同时，计划任务在后台自动拉起了一个实例）试图同时以追加模式独占打开 `telemetry.log`。虽然 `SafeStdoutWrapper` 提供了 try-except 忽略异常的自愈防护，但这说明存在多实例冲突。请通过任务管理器或 `Stop-ScheduledTask -TaskName TimeAudit_AutoStart` 停掉多余的 Python 实例即可。

装好后它就在后台默默记录，你只管开浏览器看图。数据全在本机，不上云。

PDF 从本 Markdown 生成：使用 `md-to-pdf` 技能的 `E:\.agents\plugins\md-pdf-toolkit\scripts\Invoke-MdPdfToolkit.ps1 convert --input <本文件绝对路径> --output <快速部署.pdf绝对路径>`。